

Question description

Malar, subash, and Yasir alias Pari are three first-year engineering students of the State Technical Institution (STI), India. While Malar and subash are average students who come from a Middle class, Yasir is from a rich family. Malar studies, engineering as per his father's wishes, while subash, whose family is poor, studies engineering to improve his family's financial situation.

Yasir, however, studies engineering of his simple passion for developing android applications.

Question No:8

Yasir is participating in a hackathon for android application development. the task is foling the linked list and also need to follow the given procedure.

- Firstly, Initialize a linked list.
- Then find the middle point using two pointers at different speeds through the sequence of values until they both have equal values.
- Divide the linked list into two halves using these pointer named as slow and fast, then find middle.
- Now reverse the second half of the linked list.
- At last alternate merge of first and second halves.
- Print the fold linked list.

Constraints

 $0 < N < 100$ $0 < arr < 1000$

Input Format

First line indicates the number of elements N to be inserted in array

Second line indicates the array elements according to the N

Output Format

First line represents the d linked list in original order

Second Line represents the linked list in after folding

▼ Logical Test Cases

Test Case 1

INPUT (STDIN)

16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17

EXPECTED OUTPUT

Link list data:1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
Link list data after fold:1 17 3 98 4 32 7 56 9 24 11 23 13 19 14 18

Test Case 2

INPUT (STDIN)

15
0 1 2 0 1 1 1 0 2 0 2 2 0 1 2

EXPECTED OUTPUT

Link list data:0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
Link list data after fold:0 2 1 1 2 0 0 2 1 2 1 0 1 2 0

▼ Mandatory Test Cases

Test Case 1

KEYWORD

struct node

Test Case 2

KEYWORD

void print(struct node *head)

Test Case 3

KEYWORD

struct node *next;

Test Case 4

KEYWORD

void create(struct node **head,int data)

▼ Complexity Test Cases

Test Case 1

Test Case 2

Test Case 3

Code:

```
1  #include <stdio.h>
2  #include <stdlib.h>
3
4  struct Node
5  {
6      int data;
7      struct Node *next;
8  };
9
10 struct Node *createNode(int data)
11 {
12     struct Node *newNode = (struct Node *)malloc(sizeof(struct Node));
13
14     newNode->data = data;
15     newNode->next = NULL;
16
17     return newNode;
18 }
19
20 void insertEnd(struct Node **head, int data)
21 {
22     struct Node *newNode = createNode(data);
23
24     if (*head == NULL)
25     {
26         *head = newNode;
27         return;
28     }
```

```
30     struct Node *temp = *head;
31
32     while (temp->next != NULL)
33     {
34         temp = temp->next;
35     }
36
37     temp->next = newNode;
38 }
39
40 void display(struct Node *head)
41 {
42     struct Node *temp = head;
43
44     while (temp != NULL)
45     {
46         printf("%d", temp->data);
47
48         if (temp->next != NULL)
49             printf(" ");
50
51         temp = temp->next;
52     }
```

```
54     printf("\n");
55 }
56
57 struct Node *reverse(struct Node *head)
58 {
59     struct Node *prev = NULL;
60     struct Node *current = head;
61     struct Node *next;
62
63     while (current != NULL)
64     {
65         next = current->next;
66         current->next = prev;
67         prev = current;
68         current = next;
69     }
70
71     return prev;
72 }
73
74 struct Node *fold(struct Node *head, int n)
75 {
```

```
76     if (head == NULL || head->next == NULL)
77         return head;
78
79     struct Node *first = head;
80     struct Node *second;
81     struct Node *slow = head;
82     struct Node *fast = head;
83
84     /*
85      Find the middle.
86      For even N:
87      first half = N/2
88      second half = N/2
89
90      For odd N:
91      keep the middle node with the first half.
92     */
93
94     if (n % 2 == 0)
95     {
96         while (fast->next != NULL && fast->next->next != NULL)
97         {
98             slow = slow->next;
99             fast = fast->next->next;
```

```
100     }
101
102     second = slow->next;
103     slow->next = NULL;
104 }
105 else
106 {
107     while (fast->next != NULL && fast->next->next != NULL)
108     {
109         slow = slow->next;
110         fast = fast->next->next;
111     }
112
113     second = slow->next;
114     slow->next = NULL;
115 }
116
117 /* Reverse second half */
118 second = reverse(second);
119
120 /* Alternate merge */
121 struct Node *result = NULL;
122 struct Node *tail = NULL;
123
```

```
124     while (first != NULL || second != NULL)
125     {
126         if (first != NULL)
127         {
128             struct Node *nextFirst = first->next;
129
130             if (result == NULL)
131             {
132                 result = first;
133                 tail = first;
134             }
135             else
136             {
137                 tail->next = first;
138                 tail = first;
139             }
140
141             first = nextFirst;
142         }
143
144         if (second != NULL)
145         {
146             struct Node *nextSecond = second->next;
```

```
147
148         tail->next = second;
149         tail = second;
150
151         second = nextSecond;
152     }
153 }
154
155     tail->next = NULL;
156
157     return result;
158 }
159
160 int main()
161 {
162     int N;
163     scanf("%d", &N);
164
165     struct Node *head = NULL;
166
167     for (int i = 0; i < N; i++)
168     {
169         int data;
170         scanf("%d", &data);
```

```

171
172         insertEnd(&head, data);
173     }
174
175     printf("Link list data:");
176     display(head);
177
178     head = fold(head, N);
179
180     printf("Link list data after fold:");
181     display(head);
182
183     return 0;
184 }

```

Output:

Test Case-1:

```

16
1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
Link list data:1 3 4 7 9 11 13 14 18 19 23 24 56 32 98 17
Link list data after fold:1 17 3 98 4 32 7 56 9 24 11 23 13 19 14 18

```

Test Case-2:

```

15
0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
Link list data:0 1 2 0 1 1 1 0 2 0 2 2 0 1 2
Link list data after fold:0 2 1 1 2 0 0 2 1 2 1 0 1 2 0
PS C:\Users\keshav\OneDrive\Documents\3sem\C>

```

Keshav Kumar Rauniyar
CH.SC.U4CSE25236